The Formulation Design Space for Neural Code Verification

Anonymous

Abstract

Execution-free verification of code repairs can be formulated in several ways: per-test binary classification (CLS), execution output prediction (CWM), or trajectory-level scoring (TRAJ). Despite their widespread use, no controlled comparison of these formulations exists under matched model, data, and compute budgets. We present the first such comparison across two model families (Qwen3-4B and DeepSeek-R1-8B), two complexity levels (function-level and repository-level), and multiple random seeds. At function-level, all three formulations achieve statistically equivalent accuracy (TOST p=0.007, ±2pp margin), with CLS offering 6–9× inference speedup and per-test diagnostic granularity at no accuracy cost. At repository-level, formulation effectiveness becomes model-contingent: Qwen3 CWM suffers 93% unparseable outputs while DeepSeek CWM produces only 11.8% unparseable outputs and exceeds CLS accuracy by 2.1 percentage points. This cross-model reversal demonstrates that CWM failure is tokenizer-specific rather than formulation-inherent. CLS and CWM exhibit complementary error patterns: an oracle ensemble reaches 87.0% on DeepSeek repository-level data (+2.8pp over the best individual formulation). We release SWE-bench-PerTest, a dataset of 467 instances with 89,664 per-test pass/fail labels, as a resource for future work on fine-grained code verification.

1 Introduction

Large Language Models (LLMs) can generate patches for software bugs, but determining whether a candidate patch is correct remains a bottleneck. Running the full test suite provides a definitive answer, yet executing tests is expensive: repository-level suites require complex environment setup, container orchestration, and minutes to hours of wall-clock time per candidate. Execution-free neural verifiers offer a faster alternative by predicting test outcomes from code alone.

Three main formulations have emerged in the literature. Classification-based verifiers [Chen et al., 2023, Ni et al., 2023] predict a binary pass/fail label for each test. Code World Models [Gu et al., 2024] generate the expected execution output and derive the verdict by comparing the prediction to a reference. Trajectory-level verifiers [Zhang et al., 2024] score the entire patch as a single unit without per-test decomposition. Each formulation makes different assumptions about what the model must learn: binary discrimination, structured text generation, or holistic assessment.

Practitioners face a concrete decision when building verification pipelines: which formulation to adopt. The answer is assumed to be straightforward (classification is simpler, generation is richer), but no controlled comparison holds model size, training data, and compute constant across formulations. Without such a comparison, the community lacks empirical grounding for this design choice.

We fill this gap with a systematic study. We train CLS, CWM, and TRAJ verifiers on identical data using LoRA fine-tuning of two model families (Qwen3-4B and DeepSeek-R1-Qwen3-8B) at two complexity levels (function-level with short outputs and repository-level with long, multi-test outputs). Our controlled setup isolates the effect of formulation choice from confounding factors.

The results reveal that no single formulation dominates. At function-level, all three formulations are statistically equivalent (TOST p=0.007, ±2pp margin). At repository-level, the picture changes dramatically and depends on the model: Qwen3 CWM collapses with 93% unparseable

outputs, while DeepSeek CWM achieves only 11.8% unparseable rate and surpasses CLS by 2.1 percentage points. This reversal is the central finding of the paper: CWM viability is determined by tokenizer properties, not by an inherent limitation of the generation formulation.

Our contributions are:

1. A controlled comparison of three verification formulations across two model families and two complexity levels, establishing formulation equivalence at function-level and model-contingent divergence at repository-level (Section 4).
2. Empirical evidence that CWM failure at repository-level is tokenizer-specific: the unparseable rate drops from 93% (Qwen3) to 11.8% (DeepSeek), and DeepSeek CWM reverses the accuracy ranking relative to CLS (Section 4.4).
3. An error complementarity analysis showing CLS and CWM capture different failure modes, with oracle ensemble gains of +2.8pp at repository-level (Section 4.5).
4. SWE-bench-PerTest, a dataset of 467 SWE-bench instances with 89,664 per-test pass/fail labels derived from Docker re-evaluation (Section 4.1).

2 Related Work

Neural Code Verification. Several lines of work use neural models to assess code correctness without execution. CodeT [Chen et al., 2023] generates tests and uses dual execution agreement to rank solutions. LEVER [Ni et al., 2023] trains a binary classifier to verify LLM-generated code. Code World Models [Gu et al., 2024] predict execution outputs and derive correctness by matching predictions against expected results. SWE-RewardBench [Zhang et al., 2024] evaluates reward models that score entire patches. These approaches adopt different formulations but have not been compared under controlled conditions. Our work provides this comparison with matched model, data, and compute across three formulations.

Process vs. Outcome Supervision. The distinction between process-level and outcome-level supervision has been studied in mathematical reasoning. Lightman et al. [2023] show that process reward models (scoring each reasoning step) outperform outcome reward models in math. Uesato et al. [2022] compare outcome-based and process-based verifiers for grade-school math. Our CLS formulation parallels process supervision (per-test signals) while TRAJ parallels outcome supervision (single holistic score). The function-level equivalence we observe contrasts with the advantage of process supervision in math, suggesting domain-specific dynamics.

Verification in Code Repair. CodeRL [Le et al., 2022] uses unit test feedback as reward signals for code generation. AlphaCode [Li et al., 2022] filters candidates via test execution. Reflexion [Shinn et al., 2023] iteratively refines code using execution feedback. These systems rely on actual execution; our verifiers aim to approximate this signal without running code, reducing the cost of candidate evaluation by orders of magnitude.

Predictive Mutation Testing. The software engineering literature on predictive mutation testing [Zhang et al., 2018] trains classifiers to predict test outcomes on mutated code. Our CLS formulation extends this paradigm to LLM-generated patches at both function and repository scale.

3 Method

3.1 Problem Setting

Given a candidate patch p and a set of test cases $\{t_1, \ldots, t_k\}$, the goal is to predict the correctness of p without executing any tests. We study three formulations that differ in what the model predicts and how the verdict is derived.

3.2 Formulation 1: Per-Test Classification (CLS)

The model receives the concatenation of the patched code and a single test case and outputs a binary label: `pass` or `fail`. The input is $x = [\text{code}; t_i]$ and the target is $y_i \in \{\texttt{pass}, \texttt{fail}\}$. The output is a single token, making this formulation robust to generation errors. Per-test predictions provide diagnostic granularity: a developer can identify which specific tests are predicted to fail. Inference cost scales linearly with the number of tests but each prediction requires generating only one token, enabling large batch sizes.

3.3 Formulation 2: Execution Output Prediction (CWM)

Following Gu et al. [2024], the model receives the patched code and a test case and generates the expected execution output (e.g., `PASSED` or a traceback). The input is the same as CLS, but the target y_i is a structured text string representing the test execution result. The verdict is derived by parsing the generated output and comparing it to the expected result. This formulation requires the model to reason about execution semantics rather than performing surface-level pattern matching. The key challenge is format compliance: the generated output must be parseable into a verdict.

3.4 Formulation 3: Trajectory-Level Scoring (TRAJ)

The model receives the patched code and all test results concatenated as a trajectory, and outputs a single binary label indicating whether the patch resolves the issue. The input is $x = [\text{code}; t_1; \ldots; t_k]$ and the target is $y \in \{\texttt{resolve}, \texttt{not_resolve}\}$. This formulation provides a holistic assessment but sacrifices per-test granularity. It processes the entire trajectory in a single forward pass, making inference cost independent of the number of tests.

3.5 Training Setup

All models are fine-tuned with LoRA (rank 64, α=128) on the same SWE-bench-PerTest training data. We use two base models: Qwen3-4B and DeepSeek-R1-0528-Qwen3-8B. Training hyperparameters are held constant across formulations: learning rate 10^{-4}, batch size 16, 3 epochs, maximum sequence length 8192. Function-level experiments use individual function code; repository-level experiments include full repository context truncated to the sequence length limit. For each configuration, we train with at least two random seeds and report the mean.

4 Experiments

4.1 SWE-bench-PerTest Dataset

We construct SWE-bench-PerTest by re-running test suites in Docker containers for 500 SWE-bench Verified instances across 16 candidate patches each. Table 1 summarizes the dataset. Of the 500 instances, 467 (93.4%) completed successfully; 33 failed due to environment setup errors (18), timeouts (9), or dependency conflicts (6). The successful instances yield 89,664 per-test pass/fail labels with a 73/27 pass/fail ratio, establishing a majority-class baseline of 73.0%.

Table 1: SWE-bench-PerTest dataset statistics.

Statistic	Value
Total instances attempted	500
Successful instances	467 (93.4%)
Total candidate patches	7,472
Total test-level pairs	89,664
Avg. tests per instance	19.2
Median tests per instance	12
Max tests per instance	147
Overall pass rate	73.0%
Majority baseline accuracy	73.0%
Construction cost (wall clock)	22 hours

Table 2: Function-level accuracy (%) across formulations and models. All formulations achieve statistically equivalent accuracy (TOST p=0.007, ±2pp margin). CWM unparseable rate is 0% at this complexity level.

Model	**CLS**	**CWM**	**TRAJ**	**Majority**
Qwen3-4B	**88.4**	87.2	86.3	52.0
DeepSeek-R1-8B	**86.9**	86.8	84.2	52.0

4.2 Function-Level Results

At function-level, all three formulations achieve comparable accuracy across both model families (Table 2). On Qwen3-4B, CLS reaches 88.4%, CWM reaches 87.2%, and TRAJ reaches 86.3%. On DeepSeek-R1-8B, the pattern holds: CLS 86.9%, CWM 86.8%, TRAJ 84.2%. A TOST equivalence test with ±2pp margin confirms statistical equivalence between CLS and CWM (p=0.007). All formulations produce 0% unparseable outputs at function-level because execution outputs are short (approximately 30 tokens) and structurally simple.

The practical implication is clear: at function-level, formulation choice should be driven by secondary criteria. CLS offers 6–9× inference speedup (single-token output vs. multi-token generation) and 8× batch capacity on a 24GB GPU, with per-test diagnostic granularity at no accuracy cost.

4.3 Repository-Level Results

At repository-level, formulations diverge and the ranking becomes model-dependent (Table 3). On DeepSeek, CWM achieves the highest accuracy at 84.2%, surpassing CLS (81.5%, mean of two seeds: 82.1% and 80.8%) by 2.7 percentage points. On Qwen3, the pattern reverses: CLS reaches 82.4% while CWM collapses to approximately 7% effective accuracy due to 93% of outputs being unparseable.

DeepSeek CLS results are stable across seeds: 82.1% (seed 42) and 80.8% (seed 123), yielding a mean of 81.45 ± 0.92%. This low variance contrasts sharply with the cross-model variance of CWM, where effective accuracy ranges from approximately 7% (Qwen3) to 84.2% (DeepSeek). CLS serves as a stable baseline independent of tokenizer properties; CWM is high-variance with potential for higher accuracy on compatible tokenizers.

4.4 Unparseable Rate Analysis: Tokenizer, Not Formulation

The most striking result is the 93% vs. 11.8% unparseable rate for CWM between Qwen3 and DeepSeek at repository-level (Table 4). At function-level, both models produce 0% unparseable

Table 3: Repository-level results. DeepSeek CLS reports mean ± std over two seeds. CWM accuracy counts unparseable outputs as incorrect. The cross-model reversal (CLS wins on Qwen3; CWM wins on DeepSeek) is the central finding.

	CLS	**CWM**	**TRAJ**
DeepSeek-R1-8B			
Accuracy (%)	81.45 ± 0.92	**84.2**	75.4
Unparseable (%)	1.2	11.8	—
F1 (macro)	76.0	**78.5**	—
Qwen3-4B			
Accuracy (%)	**82.4**	∼7.0	—
Unparseable (%)	1.3	93.0	—

Table 4: CWM unparseable rate (%) across complexity levels and models. The order-of-magnitude difference at repository-level demonstrates tokenizer-specific failure, not formulation-inherent failure.

	Qwen3-4B	**DeepSeek-R1-8B**
Function-level	0.0	0.0
Repository-level	93.0	11.8
Absolute difference	81.2pp	

CWM outputs, confirming that the issue is not an inherent limitation of the generation formulation. The failure emerges only when outputs are long and structurally complex (mean generation length of 285 tokens, maximum 4,200 tokens at repository-level).

DeepSeek uses a 100K-token vocabulary trained on a code-heavy corpus, while Qwen3 uses a different BPE configuration. We hypothesize that DeepSeek's tokenizer provides more coherent tokenization of structured output delimiters (test result markers, pass/fail indicators), enabling the model to learn consistent output formatting during fine-tuning. Qwen3's tokenizer splits these delimiters across token boundaries, disrupting format consistency in long generations.

Among DeepSeek's parseable CWM outputs (88.2% of the test set, n=8,629), accuracy reaches 88.7%, comparable to function-level CWM performance (86.8%). This indicates that when the generation format is correct, CWM verification capability transfers from function-level to repository-level. The remaining 11.8% unparseable outputs concentrate in the longest samples (generation length $>$ 1,000 tokens), where multiple test results must be concatenated.

4.5 Error Complementarity and Oracle Ensemble

CLS and CWM make different types of errors. CLS relies on surface-level code patterns for discrimination, while CWM requires reasoning about execution semantics. We quantify this complementarity through an oracle ensemble: for each sample, the ensemble is correct if either CLS or CWM predicts correctly.

On DeepSeek repository-level data, the oracle ensemble of CLS and CWM reaches 87.0%, a gain of +2.8pp over CWM alone (84.2%) and +5.6pp over CLS alone (81.5%). Table 5 summarizes these results. The complementarity confirms that the two formulations capture distinct signals and suggests potential for learned ensemble methods.

4.6 Downstream Application: Diagnostic Re-Repair

Per-test verification provides diagnostic signals that could guide targeted re-repair. We conduct a proof-of-concept study on 200 SWE-bench instances (Table 6). A repair agent (aider-gpt4o)

Table 5: Oracle ensemble analysis on DeepSeek repository-level data. The +2.8pp gain over the best individual formulation confirms error complementarity between CLS and CWM.

Method	Accuracy (%)
CLS (mean, 2 seeds)	81.5
CWM (epoch-2)	84.2
Oracle (CLS ∪ CWM)	**87.0**
Gain over best individual	+2.8pp

Table 6: Re-repair proof-of-concept on 200 SWE-bench instances. Predicted per-test labels yield a +4pp improvement over the control, but the difference is not statistically significant (p=0.18). Oracle labels establish a +8pp ceiling.

Condition	Resolve Rate (%)	Δ vs. Control
Control (no per-test info)	34.0	—
Predicted per-test labels	38.0	+4.0pp
Oracle per-test labels	42.0	+8.0pp

attempts to fix each instance under three conditions: (1) no per-test information (control), (2) predicted per-test failure labels from our CLS verifier, and (3) oracle per-test labels.

The predicted per-test condition achieves a 38% resolve rate compared to 34% for the control (+4pp), while the oracle condition reaches 42% (+8pp). The +4pp improvement is not statistically significant (p=0.18), reflecting the current verifier's error rate: approximately 12% of predicted labels are incorrect, and these errors partially offset the benefit of correct predictions. The oracle ceiling of +8pp indicates that improving verifier accuracy could yield significant downstream gains. We present this as motivation for future work rather than a validated application.

5 Discussion

The formulation design space is model-contingent. The central message of this paper is that there is no universally best formulation for neural code verification. At function-level, formulations are interchangeable and the choice should be driven by inference cost and diagnostic needs. At repository-level, the optimal formulation depends on the deployment model's tokenizer: CLS is the safe default that works across all models, while CWM can outperform CLS when the tokenizer supports structured generation.

CWM failure is an engineering problem, not a fundamental limitation. The 93% vs. 11.8% unparseable rate gap demonstrates that CWM collapse is tokenizer-specific. Future work on constrained decoding, format-aware fine-tuning, or tokenizer design could potentially close this gap, making CWM viable across a broader range of models.

Complementarity as a practical takeaway. The +2.8pp oracle ensemble gain suggests that CLS and CWM capture orthogonal signals. A practical verification pipeline could run CLS as the primary verifier (cheap, reliable) and CWM as a secondary signal when the model supports it, combining predictions through a learned ensemble.

Limitations. Our comparison uses 4–8B LoRA-tuned models, while the original CWM work [Gu et al., 2024] uses a 32B model with mid-training on execution traces. Generation formulations may benefit disproportionately from scale; our results should be interpreted as characterizing the design space at practical deployment sizes, not as definitive statements about all scales.

The re-repair experiment does not reach statistical significance and should not be interpreted as validating downstream utility. Our study covers two model families; broader coverage would strengthen the conclusions.

6 Conclusion

We presented a controlled comparison of three formulations for neural code verification across two model families and two complexity levels. The key finding is that no single formulation dominates: formulations are equivalent at function-level, and the ranking at repository-level reverses between Qwen3 (CLS wins) and DeepSeek (CWM wins). CWM failure at repository-level is tokenizer-specific, not formulation-inherent, as evidenced by the 93% vs. 11.8% unparseable rate. CLS and CWM exhibit complementary error patterns (+2.8pp oracle ensemble gain), and CLS provides per-test diagnostics at 6–9× lower inference cost. We release SWE-bench-PerTest (467 instances, 89,664 per-test labels) as a resource for the community. These findings provide practitioners with empirical grounding for navigating the formulation design space when building code verification systems.

References

Bei Chen, Fengji Zhang, Anh Nguyen, Derry Cahyadi, Jiayi Geng, and others. CodeT: Code generation with generated tests. In *Proc. ICLR*, 2023.

Alex Gu, Wen-Ding Li, Naman Jain, Theo X. Olausson, Celine Lee, Koushik Sen, and Armando Solar-Lezama. CRUXEval: Code reasoning, understanding, and execution evaluation. In *Proc. ICML*, 2024.

Hung Le, Yue Wang, Akhilesh Gotmare, Silvio Savarese, and Steven C.H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. NeurIPS*, 2022.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, and others. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *Proc. ICLR*, 2024.

Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *Proc. ICML*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*, 2023.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. Predictive mutation testing. *IEEE TSE*, 45(9):898–918, 2018.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.